Qwen3-Coder-Next Tier 3 Part 3 Detailed Implementation Specification

Planning Layer

Principal engineering specification for implementing the planning layer of the Qwen3-Coder-Next autonomous software engineering system.

Document type	Engineering specification
Part	Part 3 - Planning Layer
Depends on	Part 1 Foundation; Part 2 Filesystem + Local Tooling
Purpose	Define exactly how to implement the planner that decomposes work into executable, verifiable steps

Implementation note
This specification is intentionally narrow. It defines the planner only, not research, coding, testing, memory, or deployment. Those systems consume planner outputs later; they do not belong in this part.

1. Purpose

Part 3 introduces the Planning Layer. Its job is to turn an incoming task request into a structured execution plan that later agents can follow without improvising the workflow. The planner sits between the foundation runtime and every downstream capability that depends on task decomposition.

- Why this part exists: large coding tasks fail when the system jumps straight from user request to code generation.
- Problem solved: it creates an explicit plan, ordered steps, dependency edges, and handoff artifacts.
- Future parts that depend on it: Research Layer, Memory System, Testing and Review, Evaluation, Failure Recovery, Benchmark Harness, and Multi-Agent Coordination.

Core rule
The planner must not implement the solution. It must produce the execution map that makes the solution buildable, auditable, and recoverable.

2. Scope

Included in this part:

- Plan request intake and normalization
- Task decomposition into ordered plan steps
- Dependency discovery between steps
- Plan quality checks and consistency validation
- Plan artifact generation for downstream agents
- Planner state updates inside runtime memory
- Planner-facing interfaces, schemas, and logs

Excluded from this part:

- Web or repository research
- Code generation and file editing
- Testing execution
- Evaluation / judge scoring
- Failure recovery strategies
- Long-term memory retrieval
- Repository knowledge graph construction

Boundary
If a feature belongs to executing or validating the solution, it is not part of Planning. The planner only decides what should happen and in what order.

3. System Overview

The planning layer receives a task request, interprets constraints, breaks the request into work units, and emits a structured plan package. That package becomes the contract for later parts.

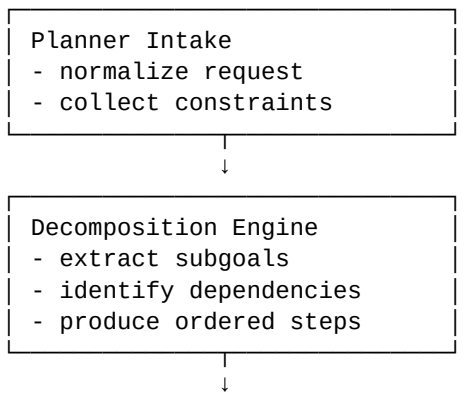
```
User Request
   ↓
LangGraph Runtime
   ↓
Planner Intake
   ↓
Goal Normalization
   ↓
Task Decomposition
   ↓
Dependency Ordering
   ↓
Plan Validation
   ↓
Plan Artifact
   ↓
Downstream Agents
```

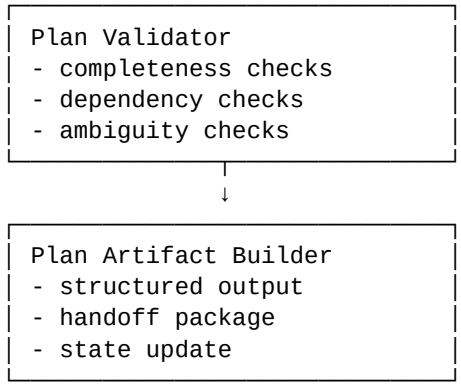
The runtime still owns orchestration. The planner owns decomposition and ordering. It does not execute tools except for the minimal metadata access needed to understand the request context.

Stage	Input	Planner action	Output
Intake	User task	Normalize intent and constraints	Plan request
Decomposition	Normalized request	Split into ordered work items	Draft steps
Validation	Draft steps	Check dependencies and completeness	Validated plan
Handoff	Validated plan	Serialize for downstream agents	Plan artifact

4. Internal Architecture

The planner should be implemented as a small set of cooperating modules. This keeps responsibility boundaries clear and makes plan generation testable without the rest of the system.





Component	Role	Consumes	Produces	Depends on
Planner Intake	Normalize task and constraints	User request, runtime context	PlannerRequest	LangGraph state
Decomposition Engine	Break the goal into steps	PlannerRequest	PlanDraft	Planner prompts, schemas
Dependency Resolver	Order steps and identify prerequisites	PlanDraft	OrderedPlan	Step graph rules
Plan Validator	Check completeness and ambiguity	OrderedPlan	ValidatedPlan	Acceptance rules
Artifact Builder	Serialize output for downstream agents	ValidatedPlan	PlanArtifact	State store, logger

Required abstractions:

- PlannerRequest: a normalized representation of the user's task and constraints.
- PlanStep: a single atomic or semi-atomic unit of work with acceptance criteria.
- PlanGraph: a directed acyclic ordering of steps and dependencies.
- PlanArtifact: the serialized handoff object consumed by later agents.
- PlanValidationReport: machine-readable defects and warnings produced by the validator.

5. Project Structure

A practical layout keeps planning code isolated from runtime orchestration and downstream agents.

```
planning/
  intake/
    request_normalizer.py
    constraint_extractor.py
  decomposition/
    goal_splitter.py
    step_builder.py
    dependency_resolver.py
  validation/
    completeness_checks.py
    ambiguity_checks.py
```

```
    ordering_checks.py
  artifacts/
    plan_schema.py
    plan_serializer.py
    handoff_package.py
  state/
    planner_state.py
    planner_events.py
  prompts/
    planner_system_prompt.md
    planner_templates.md
  policies/
    decomposition_rules.py
    acceptance_rules.py
  tests/
    test_normalization.py
    test_decomposition.py
    test_dependency_ordering.py
    test_validation.py
```

Folder responsibilities:

- planning/intake: Transforms raw user intent into a normalized planner input.
- planning/decomposition: Builds the plan itself, including step boundaries and dependency structure.
- planning/validation: Rejects incomplete, cyclic, or ambiguous plans before handoff.
- planning/artifacts: Defines the data objects and serialization format passed to downstream agents.
- planning/state: Tracks planner execution state and revision history.
- planning/prompts: Contains prompt templates and examples used by the planner logic.
- planning/policies: Holds rule sets that constrain how tasks may be split and ordered.
- planning/tests: Contains deterministic tests that prove the planner works under representative workloads.

6. Data Contracts

The planner should speak in structured objects, not free-form prose. The goal is predictable handoff and simple validation.

Contract	Required fields
PlannerRequest	task_id, user_goal, constraints, environment, source_context
PlanStep	step_id, title, objective, inputs, outputs, dependencies, acceptance_criteria, owner_hint
PlanGraph	nodes, edges, topological_order, critical_path
PlanArtifact	request_summary, ordered_steps, dependency_map, risks, assumptions, handoff_notes
ValidationReport	status, blocking_errors, warnings, coverage_metrics

PlannerEvent	event_type, timestamp, request_id, revision_id, payload

Structured examples:

```
PlannerRequest
- task_id: task-042
- user_goal: "Add feature flag support for the billing page"
- constraints: ["no database migration", "local only", "must preserve current UX"]
- environment: {"repo": "...", "branch": "feature/x"}
- source_context: ["open files", "prior discussion", "known limitations"]
```

```
PlanStep
- step_id: step-2
- title: "Define feature flag boundary"
- objective: "Specify what is behind the flag and what is unchanged"
- inputs: ["requirements", "current UI state"]
- outputs: ["flag scope", "edge cases"]
- dependencies: ["step-1"]
- acceptance_criteria: ["all impacted screens identified", "rollback path defined"]
```

7. State Management

The planner owns a dedicated slice of runtime state. That state must be sufficient to resume planning, revise a draft, and hand off cleanly to later agents.

State item	Owner	Persistence	Purpose
Current request	Planner	Session state	Defines the active planning target
Plan draft	Planner	Session state	Holds the mutable intermediate plan
Validated plan	Planner	Artifact store	Published handoff for later agents
Assumptions	Planner	Artifact store	Captures inferred but unconfirmed context
Revision history	Planner	Event log	Tracks edits to the plan over time

- State changes only when the planner emits a new revision or a validation outcome.
- Mutable draft state must be separate from immutable published artifacts.
- Downstream agents consume the validated plan, not the raw draft.
- Every revision should carry a version number so later parts can reference a stable plan snapshot.

8. Component Specifications

Request Normalizer

Purpose	Converts user input and runtime context into a planner-ready task record.
Inputs	User goal, context snippets, constraints

Outputs	PlannerRequest
Responsibilities	Enforce consistent terminology, extract hard constraints, remove ambiguity where possible
Failure conditions	Missing context, contradictory constraints
Future integration	Feeds decomposition and validation stages

Decomposition Engine

Purpose	**Turns the normalized request into candidate steps and subgoals.**
Inputs	PlannerRequest, policy rules
Outputs	PlanDraft with steps and notes
Responsibilities	Split work into executable units, preserve logical order
Failure conditions	Over-splitting, under-splitting, hidden dependencies
Future integration	Feeds dependency resolver and acceptance generation

Dependency Resolver

Purpose	**Orders steps and identifies prerequisites and blockers.**
Inputs	PlanDraft
Outputs	Ordered PlanGraph
Responsibilities	Create directed edges and a topological execution order
Failure conditions	Cycles, missing prerequisites, invalid ordering
Future integration	Feeds validator and handoff package

Plan Validator

Purpose	**Rejects incomplete or unstable plans before they reach other agents.**
Inputs	Ordered PlanGraph
Outputs	ValidationReport and/or rejection reason
Responsibilities	Check completeness, ordering, scope coverage, ambiguity
Failure conditions	False positives, overly permissive checks
Future integration	Produces gating signal for next stages

Artifact Builder

Purpose	**Serializes the final planner output in a stable format.**

Inputs	Validated plan, version metadata
Outputs	PlanArtifact
Responsibilities	Make downstream consumption deterministic and replayable
Failure conditions	Schema drift, inconsistent versioning
Future integration	Feeds Part 4 and later consumers

9. Implementation Sequence

Step 1: Define schemas and planner state

This comes first because every later module depends on stable contracts. Output: request, step, graph, artifact, and validation schemas.

Dependency: none beyond document-level design

Step 2: Build request normalization

The planner needs a predictable input shape before it can decompose anything. Output: normalized planner requests with extracted constraints.

Dependency: schemas from Step 1

Step 3: Implement decomposition rules

Once requests are normalized, the system can split them into candidate steps. Output: draft plan with subgoals and step objectives.

Dependency: normalized requests from Step 2

Step 4: Add dependency resolution

Steps must be ordered before validation can be meaningful. Output: a DAG or equivalent ordered plan graph.

Dependency: step draft from Step 3

Step 5: Add validation gates

Validation must sit after ordering so it can check completeness, ambiguity, and cycle safety. Output: pass/fail report.

Dependency: dependency-resolved plan from Step 4

Step 6: Implement artifact serialization

Later agents need a stable handoff object. Output: persisted PlanArtifact and event record.

Dependency: validated plan from Step 5

Step 7: Wire into LangGraph runtime state

The planner must update session state and emit a revisioned artifact. Output: reproducible runtime integration.

Dependency: artifact serialization from Step 6

Step 8: Add tests and sample fixtures

Regression safety is mandatory before any downstream part relies on the planner. Output: deterministic test suite and fixtures.

Dependency: all previous components

10. Validation Strategy

Validation should be layered so defects are caught as early as possible.

- Schema tests: verify every contract can be parsed, serialized, and versioned without loss of information.
- Decomposition tests: ensure a sample task produces the expected number of steps and the correct high-level split.
- Dependency tests: verify the step graph is acyclic and topologically sortable.
- Completeness tests: ensure each step has an objective, inputs, outputs, and acceptance criteria.
- Determinism tests: the same request should produce the same normalized plan unless the context changes.
- Regression tests: maintain a small curated set of feature requests that represent common planning patterns.

Verification principle
A plan is not complete because it looks reasonable. It is complete only if the schema, dependency, and acceptance checks all pass.

11. Deliverables

- Planner schema definitions
- Planner state container
- Request normalization module
- Decomposition engine
- Dependency resolver
- Validation engine
- Plan artifact serializer
- Handoff package format
- Planner unit tests and fixtures
- Runtime integration hook for LangGraph state

12. Acceptance Criteria

Criterion	How it is tested	Pass condition
Planner can decompose a feature request	Fixture request input	Outputs at least 3 coherent steps when task scope warrants it
Planner can create ordered steps	Graph validation test	Topological order exists and is stable
Planner can identify dependencies	Dependency inspection test	Each non-root step lists the prerequisite steps it requires
Planner can hand off to later agents	Artifact schema test	Produces a serialized handoff package that downstream consumers can parse
Planner captures assumptions	Annotation test	Implicit assumptions are represented explicitly in the plan artifact
Planner survives rerun on same input	Determinism test	Same input yields equivalent normalized plan revision

13. Common Failure Modes

- Over-decomposition: too many tiny steps makes the rest of the system slower and noisier.
- Under-decomposition: large vague steps push ambiguity into later parts and break execution.
- Hidden dependencies: forgetting prerequisite work leads to impossible task ordering.
- Weak validation: allowing a plan through because it sounds right instead of checking structure.
- Schema drift: changing planner output shape without versioning breaks every downstream consumer.
- Prompt leakage: relying on prose instead of structured artifacts makes the system brittle.

Design trap
The planning layer is not a place to be clever. Predictability beats elegance here, because every later system will depend on the planner's output being boring in the best possible way.

14. Future Integration

Part 3 is the contract layer for the rest of the system.

Next part	Planner output consumed	Why it matters
Part 4 Research Layer	Plan steps and assumptions	Research needs to know what facts are missing and what questions to answer
Part 5 Memory System	Plan revisions, assumptions, and recurring patterns	Memory improves future planning consistency
Part 6 Testing and Review	Acceptance criteria and step	Testing needs a target for each

	objectives	planned change
Part 7 Evaluation Layer	Plan scope and expected outcomes	Evaluation can compare final results to the intended plan
Part 8 Failure Recovery	Plan graph, blocked steps, failure notes	Recovery needs a structured representation of what failed

The planner should also emit stable identifiers so every downstream stage can reference plan steps without ambiguity. That is what makes long multi-agent runs traceable instead of chaotic.

15. Completion Checklist

- All planner schemas are defined and versioned.
- Request normalization is deterministic and test-covered.
- Plan decomposition produces structured steps, not prose.
- Dependency ordering is validated and cycle-free.
- Every step includes an objective, inputs, outputs, and acceptance criteria.
- Validated plans can be serialized into a stable handoff artifact.
- Planner state persists revisions and assumptions correctly.
- All acceptance criteria have automated tests.
- The planner integrates with the runtime without leaking execution logic.

Done means done
If the planner cannot be tested, serialized, or consumed by the next part, it is not complete. It is just a pretty outline with commitment issues.